МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Київський національний університет будівництва і архітектури
Кафедра кібербезпеки та комп`ютерної інженерії

ЗВІТ

з лабораторної роботи № 4

Розробка сенсорної ноди та публікація даних у систему

Виконав: студент групи БІКСм-26
Тарасов Максим

Перевірив: Маринський К.В

Київ – 2026

1. Опис сенсорної архітектури

Метою роботи було сформувати навички інтеграції сенсорів у ROS 2: реалізувати сенсорну ноду, організувати публікацію вимірювань у топік та застосувати до даних базову обробку (фільтрацію). Роботу виконано у віртуальній машині з Ubuntu та ROS 2 дистрибутива Lyrical, у робочому просторі ros2_ws, створеному в попередніх лабораторних роботах. Реальний сенсор не використовувався, тому вимірювання симулюються генератором випадкових значень.

Архітектура складається з однієї ноди sensor_node, яка за таймером із частотою 10 Гц виконує повний цикл обробки: генерує «сире» значення сенсора, публікує його, усереднює за ковзним вікном, публікує відфільтроване значення та порівнює його з порогом. Топіки, які використовує нода, наведено в таблиці 1.

Таблиця 1 – Топіки сенсорної ноди

Топік	**Тип повідомлення**	**Напрямок**	**Призначення**
/sensor_data	std_msgs/msg/ Float32	публікація	сирі виміри сенсора, 10 Гц
/sensor_data_filtered	std_msgs/msg/ Float32	публікація	значення після усереднення
/cmd_vel	geometry_msgs/ msg/Twist	публікація	команда зупинки при перевищенні порога

Спочатку в каталозі ~/ros2_ws/src створено пакет sensor_node типу ament_python із залежностями rclpy, std_msgs і geometry_msgs (остання потрібна для опціональної зупинки робота через /cmd_vel). Перед цим підключено середовище ROS 2 та робочий простір (рисунок 1):

```
source /opt/ros/lyrical/setup.bash
source ~/ros2_ws/install/setup.bash
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python sensor_node \
    --dependencies rclpy std_msgs geometry_msgs
```

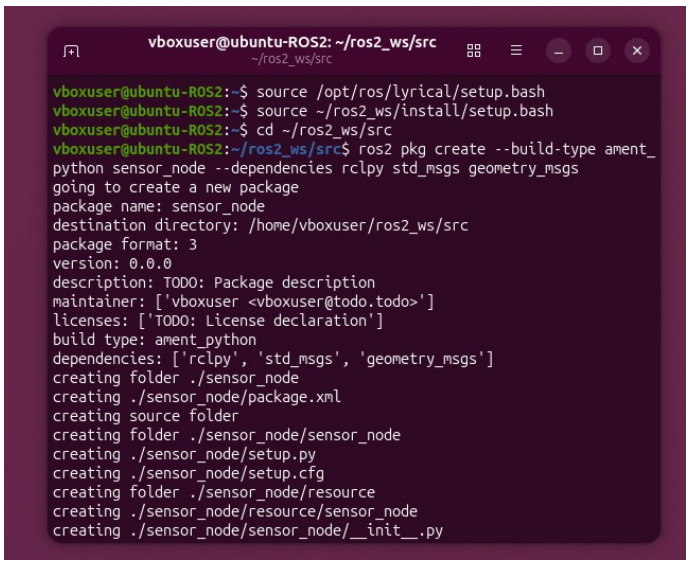


Рисунок 1 – Створення пакета sensor_node

Пакет зібрано командою colcon build --packages-select sensor_node; збірка завершилась успішно за 2.15 с (загальний час 2.56 с), після чого виконано source install/setup.bash (рисунок 2).

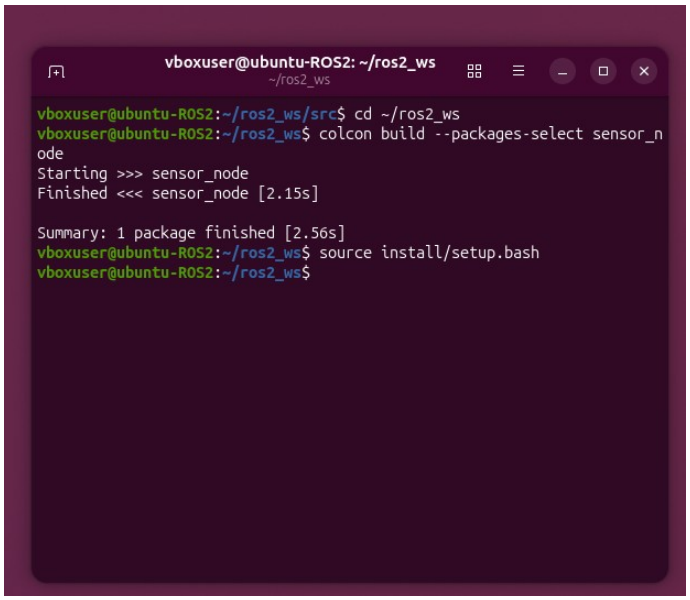


Рисунок 2 – Збірка пакета sensor_node

Щоб ноду можна було запускати командою ros2 run, у файлі setup.py розділ entry_points доповнено точкою входу sensor_node, яка вказує на функцію main модуля sensor_node.sensor_node (рисунок 3):

```
entry_points={
    'console_scripts': [
        'sensor_node = sensor_node.sensor_node:main',
    ],
},
```



Рисунок 3 – Реєстрація точки входу sensor_node у файлі setup.py

Після додавання ноди workspace зібрано повністю. До збірки увійшли чотири пакети: motion_control, my_py_pkg, robot_gui та sensor_node, усі зібрано без помилок за 3.33 с (рисунок 4).

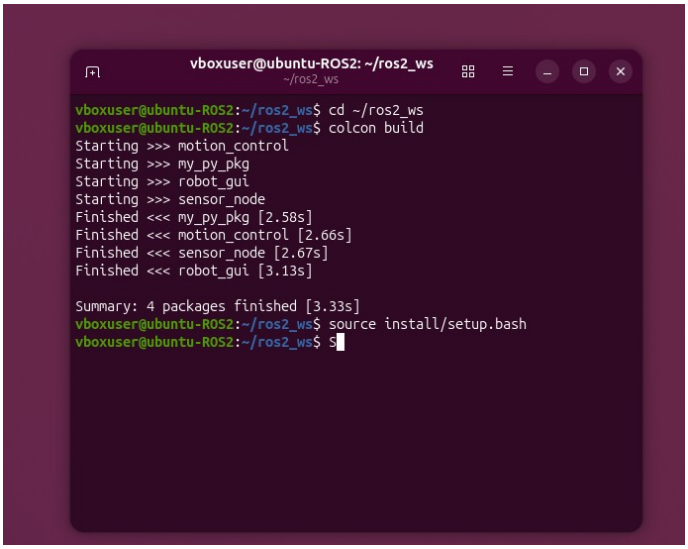


Рисунок 4 – Збірка всього workspace

2. Фрагменти коду фільтрації

Код ноди міститься у файлі ~/ros2_ws/src/sensor_node/sensor_node/sensor_node.py. У конструкторі створюються три Publisher, задаються параметри обробки (поріг threshold = 80.0 та розмір вікна усереднення window_size = 5), а також таймер з періодом 0,1 с, що забезпечує частоту 10 Гц:

```
self.raw_pub = self.create_publisher(Float32, '/sensor_data', 10)
self.filtered_pub = self.create_publisher(Float32,
'/sensor_data_filtered', 10)
self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10)

self.threshold = 80.0          # порогове значення
self.window_size = 5           # розмір вікна для усереднення
self.values = deque(maxlen=self.window_size)

self.timer = self.create_timer(0.1, self.timer_callback)   # 10 Гц
```

2.1. Симуляція сенсора

Тестове значення формується як базовий рівень 50 з випадковим шумом від –15 до +40, після чого обмежується діапазоном 0–100. Сире значення одразу публікується в топік /sensor_data:

```
raw_value = 50.0 + random.uniform(-15.0, 40.0)
raw_value = max(0.0, min(100.0, raw_value))   # обмеження 0-100

raw_msg = Float32()
raw_msg.data = raw_value
self.raw_pub.publish(raw_msg)
```

2.2. Усереднення значень

Фільтрація реалізована як ковзне середнє за останніми п’ятьма вимірами. Для цього використано чергу deque з параметром maxlen: при додаванні нового значення найстаріше автоматично видаляється. Відфільтроване значення дорівнює середньому арифметичному елементів черги й публікується в топік /sensor_data_filtered:

```
self.values.append(raw_value)
filtered_value = sum(self.values) / len(self.values)

filtered_msg = Float32()
filtered_msg.data = filtered_value
self.filtered_pub.publish(filtered_msg)
```

Усереднення згладжує різкі стрибки сирих даних: наприклад, на рисунку 6 одиничний викид 88.02 дає відфільтроване значення лише 70.00, а виміри 38.18 і 80.19 – 70.57 та 74.97 відповідно.

2.3. Перевірка порога та зупинка робота

Порівняння з порогом виконується для відфільтрованого, а не сирого значення, тому одиничні викиди не викликають хибних тривог. Якщо середнє перевищує 80.0, у журнал виводиться повідомлення рівня WARN, а метод stop_robot публікує в /cmd_vel нульову команду Twist (опціональне завдання 4.2). В іншому разі виводиться інформаційне повідомлення зі значеннями сирого та відфільтрованого сигналів:

```
if filtered_value > self.threshold:
    self.get_logger().warning(
        f'УВАГА! Перевищено поріг: {filtered_value:.2f} >
{self.threshold}'
    )
    self.stop_robot()
else:
    self.get_logger().info(
        f'Сире: {raw_value:.2f} | Відфільтроване: {filtered_value:.2f}'
    )

def stop_robot(self):
    stop_msg = Twist()
    stop_msg.linear.x = 0.0
    stop_msg.angular.z = 0.0
    self.cmd_pub.publish(stop_msg)
```

Повний код файлу sensor_node.py наведено на рисунку 5.



Рисунок 5 – Повний код сенсорної ноди (sensor_node.py)

3. Скриншоти публікації даних

Для перевірки сенсорну ноду запущено командою ros2 run sensor_node sensor_node. Нода вивела повідомлення «Sensor Node запущено (10 Гц)» і почала публікувати дані. У другому терміналі отримання вимірювань перевірено командою ros2 topic echo /sensor_data (рисунок 6, ліворуч), а праворуч наведено журнал сенсорної ноди з парою значень «сире – відфільтроване».

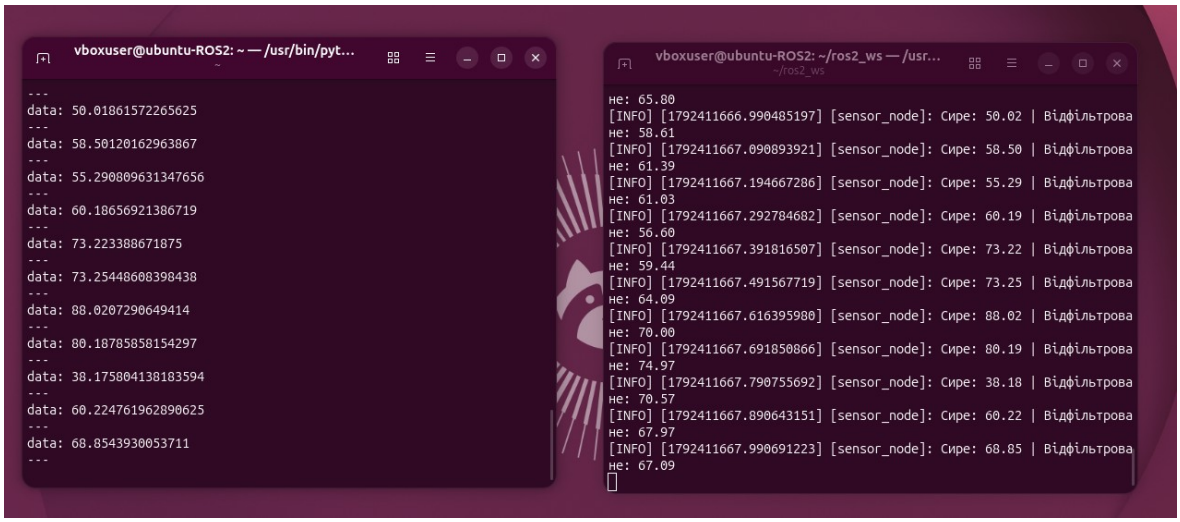


Рисунок 6 – Публікація даних: ros2 topic echo /sensor_data (ліворуч) і журнал sensor_node (праворуч)

Значення в ros2 topic echo збігаються з полем «Сире» в журналі ноди (50.02, 58.50, 55.29, 60.19, 73.22 тощо), що підтверджує коректну публікацію в топік /sensor_data. Позначки часу в журналі йдуть із кроком близько 0,1 с, тобто частота публікації становить 10 Гц, як і вимагалось.

4. Демонстрація реакції системи

Реакцію системи перевірено за журналом sensor_node (рисунок 6, праворуч). Сирі значення коливаються в широких межах (від 38.18 до 88.02), тоді як відфільтровані змінюються плавно – у межах 56.60–74.97. Оскільки середнє за п’ять вимірів у цьому фрагменті не перевищувало порога 80.0, нода виводила інформаційні повідомлення рівня INFO, а зупинка робота не вмикалась. Навіть сирі значення 88.02 і 80.19, які самі по собі перевищують

поріг, не призвели до тривоги, бо відфільтроване значення залишилось нижчим (70.00 та 74.97).

Для перехідного режиму логіка реакції така:

1. якщо відфільтроване значення стає більшим за 80.0, нода виводить у журнал повідомлення WARN «УВАГА! Перевищено поріг: <значення> > 80.0»;
2. одночасно в /cmd_vel публікується нульова команда (linear.x = 0.0, angular.z = 0.0), яка зупиняє робота; команда повторюється на кожному такті 10 Гц, доки значення залишається вище порога;
3. коли середнє знову стає не більшим за 80.0, повідомлення WARN припиняються й нода повертається до звичайного режиму INFO.

Таким чином, інші ноди системи (наприклад, Motion Node) можуть підписуватись на /sensor_data_filtered для моніторингу, а команда зупинки в /cmd_vel інтегрує сенсорну ноду з керуванням рухом без жодних змін у коді інших нод.

5. Висновки

У ході лабораторної роботи створено пакет sensor_node на Python (ament_python) із залежностями від std_msgs і geometry_msgs та розроблено сенсорну ноду, яка симулює вимірювання сенсора й публікує їх у топік /sensor_data з частотою 10 Гц. Для базової обробки даних реалізовано усереднення за ковзним вікном із п’яти значень, результат якого публікується в окремий топік /sensor_data_filtered.

Додано перевірку порога 80.0: у разі його перевищення нода виводить WARN-повідомлення та публікує в /cmd_vel нульову команду, що зупиняє робота. Отримання даних перевірено командою ros2 topic echo – значення в топіку збігаються з журналом ноди, а відфільтровані дані суттєво менш схильні до різких стрибків, ніж сирі. Опрацьовані прийоми (таймер, кілька Publisher, ковзне середнє, порогова перевірка) можна застосувати до

реальних сенсорів робота, замінивши генератор випадкових чисел драйвером пристрою.